

Solar Farm Maintenance Analysis

Question Code: Q9405

1. Problem Statement

A renewable-energy operator tracks inspection records from solar farms.

Build a Pandas analyzer that creates the inspection table, cleans invalid records, flags excessive downtime, summarizes site performance, identifies low-output sites, and calculates completed-maintenance cost by region.

Complete the class in solution.py.

Do not change the class name, method names, parameter lists, or return types.

Every analysis method receives a complete DataFrame and must work independently.

2. Input Schema

Inspection DataFrames use these columns in this exact order:

1. InspectionID
2. SiteID
3. Region
4. InspectionDate
5. OutputMWh
6. DowntimeHours
7. MaintenanceStatus

Keep InspectionDate as supplied; datetime conversion is not required.

3. Class

class SolarMaintenanceAnalyzer:

No constructor state is required.

4. Operations

Operation 1: Create the inspection DataFrame

Function

```
def create_inspections_df(
    self,
    inspection_data: list
) -> pd.DataFrame:
```

Implementation flow

- Create one row for each inner list.
- Use the seven required columns in the exact stated order.
- Preserve input row order and return a Pandas DataFrame.

Operation 2: Clean inspection data

Function

```
def clean_inspection_data(self, df: pd.DataFrame) -> pd.DataFrame:
```

Implementation flow

Retain only rows satisfying all rules:

- No column value is missing.

- OutputMWh is strictly greater than 0.
- DowntimeHours is greater than or equal to 0.
- MaintenanceStatus is either "Completed" or "Scheduled".

Return requirements

Return a copy with:

- the original seven-column order,
- retained row order, and
- a zero-based index.

Do not modify the input DataFrame.

Operation 3: Add the attention flag

Function

```
def add_attention_flag(
    self,
    df: pd.DataFrame,
    downtime_threshold: float
) -> pd.DataFrame:
```

Implementation flow

- Return a copy of df.
- Add NeedsAttention as the last column.
- Store integer 1 when DowntimeHours > downtime_threshold; otherwise store integer 0.
- Equality with the threshold must produce 0.

Operation 4: Create the site performance summary

Function

```
def site_performance_summary(self, df: pd.DataFrame) -> pd.DataFrame:
```

Implementation flow

Group by SiteID and return these columns:

- SiteID
- InspectionCount: number of inspection rows
- TotalOutputMWh: sum of OutputMWh
- AverageDowntime: mean of DowntimeHours, rounded to two decimals

Return requirements

- Sort by SiteID ascending.
- Return a zero-based index.

Operation 5: Find low-output sites

Function

```
def low_output_sites(
    self,
```

```
df: pd.DataFrame,  
output_threshold: float  
) -> pd.DataFrame:
```

Implementation flow

- Sum OutputMWh by SiteID.
- Keep sites whose total is strictly less than output_threshold.
- Equality with the threshold must not be included.

Return requirements

Return:

SiteID

TotalOutputMWh

- Sort by SiteID.
- Return a zero-based index.

Operation 6: Create the regional maintenance-cost summary

Function

```
def regional_maintenance_cost(self, df: pd.DataFrame) -> pd.DataFrame:
```

Implementation flow

- Use only rows where MaintenanceStatus == "Completed".
- Calculate row cost as DowntimeHours * 250.0.
- Sum the cost by Region.

Return requirements

Return:

Region

MaintenanceCost

- Sort by Region.
- Return a zero-based index.

Example

A completed inspection with 2.0 downtime hours contributes:

500.0

Cold-Storage Temperature Analyzer

Question Code: Q9303

1. Problem Statement

A food distribution centre records cold-room temperatures in degrees Celsius. Implement five independent NumPy functions that create, validate, summarize, categorize, and scan those readings.

Complete solution.py **without changing the function names, parameter lists, or return types.**

A valid recorded temperature is from -30.0 through 10.0, inclusive.

2. Operations

Operation 1: Create a temperature array

Function

`def create_temperature_array(values: list) -> np.ndarray:`

Implementation flow

- Convert values to a one-dimensional NumPy array.
- Return dtype must be np.float64.
- Preserve the input order.

Example

`[-22, -18.5, 4]` becomes a float64 array containing:

`[-22.0, -18.5, 4.0]`

Operation 2: Validate a temperature array

Function

`def validate_temperature_array(arr: np.ndarray) -> bool:`

Implementation flow

Return the Python boolean True only when all conditions hold:

- arr is not empty.
- Its dtype is a numeric NumPy subtype.
- Every value is from -30.0 to 10.0, inclusive.

Return False for:

- an empty array,
- a non-numeric array, or
- an out-of-range reading.

A non-numeric array must not cause an exception.

Operation 3: Compute temperature statistics

Function

`def compute_temperature_stats(arr: np.ndarray) -> tuple:`

Return value

Return:

`(mean, standard_deviation, maximum, minimum)`

Implementation flow

- Calculate the mean with `np.mean`.
- Calculate population standard deviation with `np.std` and default `ddof=0`.
- Find maximum and minimum values.
- Round all four returned values to two decimal places and return Python float values.

Example

Input:

[-20.0, -10.0, 0.0]

Returns:

(-10.0, 8.16, 0.0, -20.0)

Operation 4: Categorize temperatures

Function

def categorize_temperatures(arr: np.ndarray) -> np.ndarray:

Return a NumPy string array with one category per input value:

Category
Temperature Range
Frozen -30.0 <= value <= -18.0
Chilled -18.0 < value <= 5.0
Warning 5.0 < value <= 10.0
Invalid value below -30.0 or above 10.0

Boundary example

Input:

[-30, -18, -17.9, 5, 5.1, 10, 10.1]

Output:

[Frozen, Frozen, Chilled, Chilled, Warning, Warning, Invalid]

Operation 5: Find the longest warning streak

Function

def longest_warning_streak(arr: np.ndarray) -> int:

Implementation flow

- A warning reading satisfies $5.0 < \text{value} \leq 10.0$.
- Return the greatest number of consecutive warning readings.
- Any other value, including an invalid value, breaks the current streak.
- Return 0 when no warning reading exists.

Museum Artifact Loan Registry

Question code: Q9203

1. Problem Statement

A museum records artifacts temporarily loaned to external institutions. Build a small in-memory registry that can register a loan, change its destination, look up one loan, and list loans sent to a destination.

Complete the class in `solution.py`. Do not change the class name, state attribute, method names, parameter lists, or return types.

2. Class and State

```
class MuseumLoanRegistry:
```

```
    def __init__(self):
```

```
        self.loans = {}
```

`self.loans` maps each `loan_id` to a dictionary with exactly these keys:

```
{  
    "artifact_name": str,
```

```
"destination": str,  
"status": "On Loan"  
}
```

3. Operations

Operation 1: Register an artifact loan

```
def register_loan(  
    self,  
    loan_id: str,  
    artifact_name: str,  
    destination: str  
) -> dict:
```

Purpose: add a new loan to the registry.

Implementation flow:

- Check whether loan_id already exists.
- If it exists, raise ValueError("Loan already registered") without changing the stored record.
- Otherwise store the artifact name, destination, and status "On Loan".
- Return the complete updated self.loans dictionary.

Example:

```
registry.register_loan("L201", "Bronze Lamp", "City Gallery")
```

Expected result:

```
{  
    "L201": {  
        "artifact_name": "Bronze Lamp",  
        "destination": "City Gallery",  
        "status": "On Loan"  
    }  
}
```

Operation 2: Change a loan destination

```
def change_destination(  
    self,  
    loan_id: str,  
    new_destination: str  
) -> dict:
```

Implementation flow:

- If loan_id does not exist, raise KeyError("Loan not found").
- Update only the destination field of the matching loan.
- Preserve its artifact name and status.
- Return the complete updated self.loans dictionary.

Operation 3: Get loan details

```
def get_loan_details(self, loan_id: str) -> dict:
```

- If loan_id does not exist, raise KeyError("Loan not found").

- Return the stored details dictionary for that loan.

Operation 4: List loans by destination

def loans_by_destination(self, destination: str) -> list:

- Return loan IDs whose stored destination exactly equals destination.
- Matching is case-sensitive.
- Preserve the insertion order of self.loans.
- Return an empty list when there are no matches.

SQL

SQL NEW QUESTION 2

SQL Question: Event Average Finish Time

Marks: 20

Difficulty: Medium

Database: ATHLETICS_RESULTS_DB

Schema Details

1. Events Table



Column Name	Data Type	Description
event_id	INT (PK)	Unique event ID
event_name	VARCHAR(100)	Name of the event

2. Athletes Table

Column Name	Data Type	Description
athlete_id	INT (PK)	Unique athlete ID
athlete_name	VARCHAR(100)	Name of the athlete

3. Results Table

Column Name	Data Type	Description
result_id	INT (PK)	Unique result ID
event_id	INT (FK)	Event in which the athlete participated
athlete_id	INT (FK)	Athlete who participated
finish_time	DECIMAL(5,2)	Finish time in seconds
status	ENUM('Recorded','Disqualified','DNS')	Status of the result

Sample Data



Events

event_id	event_name
1	100m Sprint
2	200m Sprint

Athletes

athlete_id	athlete_name
1	Alice
2	Brian
3	Carlos
4	Diana

Results					▼
result_id	event_id	athlete_id	finish_time	status	
201	1	1	11.20	Recorded	
202	1	2	10.95	Recorded	
203	1	3	12.10	Recorded	
204	1	4	11.50	Disqualified	
205	2	1	22.50	Recorded	
206	2	2	21.80	Recorded	
207	2	3	21.80	Recorded	
208	2	4	23.10	DNS	

Problem Statement

A national athletics federation stores athlete performance data for multiple track events.

Generate a report that, for every recorded result, displays:

- The event name
- The athlete name
- The athlete's finish time in seconds
- The average finish time of all recorded performances within the same event

Use the AVG() window function with PARTITION BY to calculate the average finish time separately for each event.

Only results where status = 'Recorded' should be included.

Results with statuses such as Disqualified or DNS must not appear in the final output and must not be included while calculating the event average.

Round the calculated average finish time to 2 decimal places.

Sort the final result by:

1. event_name
2. finish_time
3. athlete_name

Required Output Columns

Expected Output			
event_name	athlete_name	finish_time	avg_event_time
100m Sprint	Brian	10.95	11.42
100m Sprint	Alice	11.20	11.42
100m Sprint	Carlos	12.10	11.42
200m Sprint	Brian	21.80	22.03
200m Sprint	Carlos	21.80	22.03
200m Sprint	Alice	22.50	22.03

- event_name
- athlete_name
- finish_time
- avg_event_time

SQL NEW QUESTION 1

SQL Question: Days Between Customer Orders

Marks: 20

Database: CUSTOMERS_ORDERS_DB

Schema Details

1. Customers Table



Column Name	Data Type	Description
customer_id	INT (PK)	Unique customer ID
customer_name	VARCHAR(100)	Customer name

2. Orders Table

Column Name	Data Type	Description
order_id	INT (PK)	Unique order ID
customer_id	INT (FK)	Customer who placed the order
order_date	DATE	Date when the order was placed
amount	DECIMAL(10,2)	Order amount

Sample Data



Customers

customer_id	customer_name
1	Aarav
2	Diya
3	Kabir

Orders

order_id	customer_id	order_date	amount
501	1	2025-08-02	800.00
506	1	2025-08-07	1200.00
514	1	2025-08-19	950.00
602	2	2025-08-04	650.00
609	2	2025-08-13	1050.00
701	3	2025-08-06	1500.00
708	3	2025-08-16	1100.00

Problem Statement

For each order placed by a customer, calculate the number of days that have passed since the customer's immediately previous order.

Use the LAG() window function to retrieve the previous order date for the same customer.

Requirements

- Process the orders separately for each customer using customer_id.
- Use LAG() to retrieve the immediately previous order_date for each customer.
- Determine the sequence of orders using order_date and then order_id.
- Calculate the number of days between the current order date and the previous order date.
- For the first order of each customer, days_since_previous_order should be NULL.
- Sort the final result by:
 1. customer_name
 2. order_date
 3. order_id

Required Output Columns

- customer_name
- order_id
- order_date
- days_since_previous_order

Expected Output

customer_name	order_id	order_date	days_since_previous_order
Aditi	101	2025-11-07T00:00:00.000Z	NULL
Aditi	102	2025-11-11T00:00:00.000Z	4
Aditi	105	2025-11-15T00:00:00.000Z	4
Bharat	201	2025-11-14T00:00:00.000Z	NULL